

EEE2045F Module 2 Notes

Jarryd Son

The following notes contain information regarding supervised learning.

We will begin our descent into machine learning with a more thorough exploration of supervised learning scenarios. In the following section we will look at how we can use linear regression to solve a regression task using a supervised learning approach. It is great for its simplicity and interpretability (easy to see which features are important based on their learned coefficient), although the assumption that the data fits a linear model is rather limiting. These types of algorithms that assume the solution has a specific form belong to a class of methods called *parametric* methods ¹.

The Supervised Learning Process

Here we will formalize the whole supervised learning process. In a supervised learning scenario we have access to some labelled dataset consisting of N examples denoted:

$$D = \{(\mathbf{x}^{[i]}, \mathbf{y}^{[i]}), \dots, (\mathbf{x}^{[n]}, \mathbf{y}^{[n]})\}$$

Where $\mathbf{x}^{[i]}$ is the i^{th} example with a corresponding ground truth label $\mathbf{y}^{[i]}$.

The goal is to learn some function, which we will call a “hypothesis” function, that predicts target values using examples (represented by vectors/tensors of features) from the dataset as inputs. This gives us:

$$h(\mathbf{x}) = \mathbf{y}$$

Where subscript $\cdot$ is used to indicate that the hypothesis function is parameterised by some set of values, which we will call $\cdot$. We will need to learn these parameters in order to maximize/minimize some objective function.

The process can be visualised with an informal a block diagram as shown in Figure 1.

The learning phase is often repeated a number of times ². The model can be tested on the holdout test set to determine whether or not it is appropriate for use on external data. If the model does not test well it is likely you will have to make modifications to the data, learning algorithm etc. in order to improve this. Once you are satisfied with the model you can save its parameters and deploy it on the real world.

¹ Chapter 2 of [1] as well as [2] provide good explanations of parametric and non-parametric models

[1] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An introduction to statistical learning*. Springer, 2017; and Sebastian Raschka. What is the difference between a parametric learning algorithm and a nonparametric learning algorithm?, 2020. URL https://sebastianraschka.com/faq/docs/parametric_vs_nonparametric.html

² depends on the algorithm, but many of the ones we will look at have an iterative approach

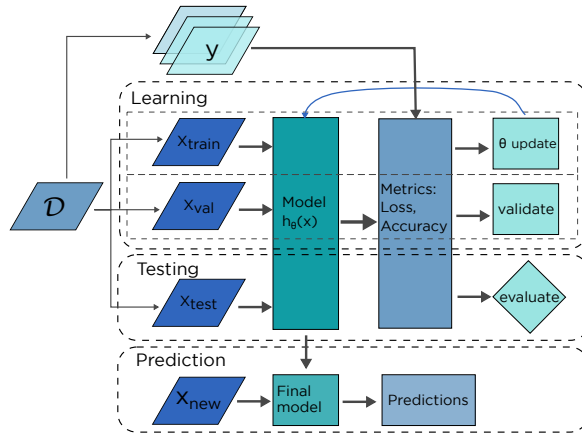


Figure 1: A block diagram of the supervised learning process

Simple Supervised Learning Algorithms

Now that we have an idea of what supervised learning entails we can explore some basic examples.

Linear Regression

Linear regression is often used as a starting base for understanding supervised learning. It is easy to understand and implement, but makes some assumptions that may not hold true for many real datasets. For simple linear regression³ we will attempt to predict a single response, y , from a single predictor variable (feature), x . As the name implies a linear regression model assumes that there is an approximately linear relationship between the response and predictor variable, such that the hypothesis function that we will attempt to learn has the form:

$$h_{\theta}(x) = y \approx \theta_0 + \theta_1 x$$

The goal is to learn the estimates of the parameters $\hat{\theta}_0$ and $\hat{\theta}_1$ that optimize some objective function that relates the ground truth value, y , with the predicted value from our algorithm, $\hat{y}$.

Suppose we have collected data regarding the battery life of a smartphone compared to its screen size⁴. Battery life would be the response we are trying to predict, and screen size would be the feature used to predict it. There are of course other things that likely impact battery life, such as the kinds of apps the user runs and the length of time they use it for, but who knows, maybe just screen size alone gives us a good estimate. You could see the linear regression problem as:

$$\text{battery_life} \approx \theta_0 + \theta_1 \times \text{screen_size}$$

³ Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An introduction to statistical learning*. Springer, 2017

⁴ Being electrical engineers we could of course look through datasheets to figure out current draw estimates for the components and use that to estimate battery life, but that could take a fair bit of effort and wouldn't be much fun

From Section ?? we know that we need to choose an objective function and a method to optimize that objective function with respect to our parameters. Let's choose our objective function to be the Mean Squared Error function given by:

$$\begin{aligned} J(\theta_0, \theta_1) &= \frac{1}{N} \sum_{i=1}^N (h_{\theta}(x_i) - y_i)^2 \\ &= \frac{1}{N} \sum_{i=1}^N ((\theta_0 + \theta_1 x_i) - y_i)^2 \\ &= \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2 \end{aligned}$$

Where $\hat{y}_i$ is the predicted value from our hypothesis function parameterised by θ .

It turns out that it's fairly simple to find an analytical solution to this specific problem, however, we will use a gradient descent approach, as it will be used extensively in many of the algorithms we will cover in this course.

Gradient Descent

Performing optimization by gradient descent is quite elegant in its simplicity and is quite intuitive. We can try find the gradient of a given loss function with respect to each of its parameters. If you can calculate the gradient you could change the parameter to move in the opposite direction to the gradient. Doing so multiple times should⁵ move the position on the loss curve closer and closer towards a minimum. See the illustration in Figure ??.

⁵ many caveats that we will discuss at a later stage

Here we will introduce a hyperparameter known as the learning rate, η which scales how much we move the parameter in the opposite direction to the gradient. This gives us our first learning rule that can be formalised as:

$$\theta_j := \theta_j - \eta \frac{\partial J(\cdot)}{\partial \theta_j} \quad (1)$$

Where “:=” is an assignment operation.

So let's find the expressions for the gradients (partial derivatives) for each parameter with respect to the objective function.

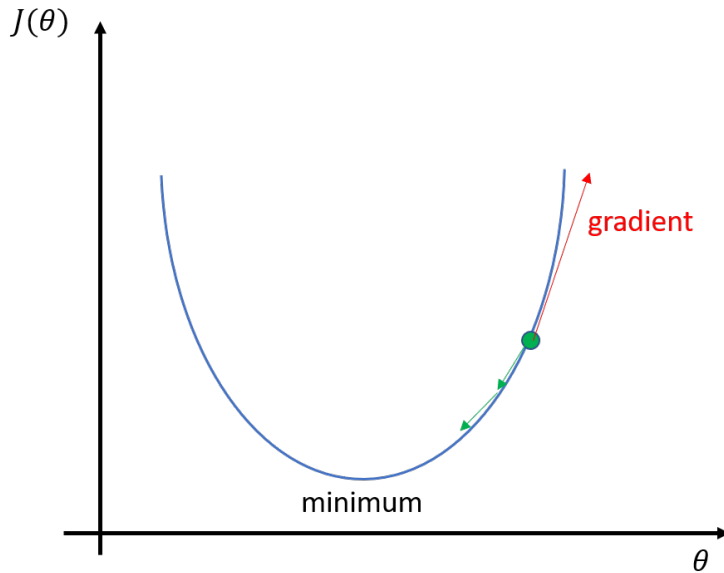


Figure 2: Successive movements in the opposite direction to the gradient would lead to a minimum

For θ_0

$$\begin{aligned}\frac{\partial J(\theta)}{\partial \theta_0} &= \frac{\partial}{\partial \theta_0} \left[\frac{1}{N} \sum_{i=1}^N ((\theta_0 + \theta_1 x_i) - y_i)^2 \right] \\ &= \frac{2}{N} \sum_{i=1}^N \left[((\theta_0 + \theta_1 x_i) - y_i) \cdot \frac{\partial}{\partial \theta_0} [(\theta_0 + \theta_1 x_i) - y_i] \right] \\ &= \frac{2}{N} \sum_{i=1}^N [((\theta_0 + \theta_1 x_i) - y_i) \cdot 1]\end{aligned}$$

For θ_1

$$\begin{aligned}\frac{\partial J(\theta)}{\partial \theta_1} &= \frac{\partial}{\partial \theta_1} \left[\frac{1}{N} \sum_{i=1}^N ((\theta_0 + \theta_1 x_i) - y_i)^2 \right] \\ &= \frac{2}{N} \sum_{i=1}^N \left[((\theta_0 + \theta_1 x_i) - y_i) \cdot \frac{\partial}{\partial \theta_1} [(\theta_0 + \theta_1 x_i) - y_i] \right] \\ &= \frac{2}{N} \sum_{i=1}^N [((\theta_0 + \theta_1 x_i) - y_i) \cdot x_i]\end{aligned}$$

Once the gradients have been determined it is possible to perform the parameter update from 1 We can repeat this updating process until we have reach satisfactory values for the objective function, or we can choose to stop after a certain number of passes over the dataset ⁶

Figure 3 is to remind you of the different tasks and scenarios. We will continue our study of supervised learning with algorithms that solve classification tasks.

⁶ The term *epoch* describes one pass over all examples

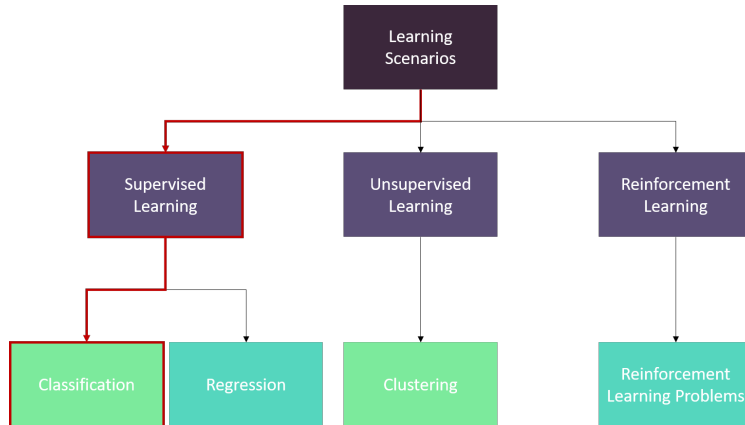


Figure 3: Review of the learning tasks and scenarios. We are going to look at a simple algorithm that solves classification tasks next

The next algorithm we will look at is the k-Nearest Neighbour algorithm.

k-Nearest Neighbour Algorithm

The k-Nearest Neighbour (kNN) ⁷ algorithm is quite a different approach to learning. Rather than looking through all the training data to produce a model that will then predict an output, the kNN algorithm stores the training data and uses it as a database to compare with incoming unseen data. This type of learning is rather aptly named *lazy* learning, as we do not actually learn anything until we are tested. The alternative to this is *eager* learning where the learner digests the training data to learn a model which is then used to predict unseen test data.

Think of this as a *lazy* approach to learning for an open-book test. You would have access to content to learn from (training data), however, seeing as it is open-book you decide to answer the test by comparing the test questions with the training content you have access to. Compare this to learning the content before being tested, as was the case for linear regression.

It is a very simple approach, but can be very useful in certain cases. It is typically used for classification ⁸ but can easily be modified to solve regression tasks. The general idea for a kNN is to determine which training examples an unseen test example is closest (similar) to. Once this has been determined, a voting process is used to classify the new test example. For example see Figure 4

To formalize the algorithm ⁹ we have some labelled training dataset:

$$D = \{(\mathbf{x}^{[i]}, \mathbf{y}^{[i]}), \dots, (\mathbf{x}^{[n]}, \mathbf{y}^{[n]})\}$$

⁷ not be confused with the abbreviation for neural networks which we will look at later

⁸ We will focus on its use for classification.

⁹ See Chapter 2.2.3 of [1] and [2] for more details

Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An introduction to statistical learning*. Springer, 2017; and Sebastian Raschka. Stat 479: Machine learning lecture notes. <http://pages.stat.wisc.edu/~sraschka/teaching/stat479-fs2019>, 2019

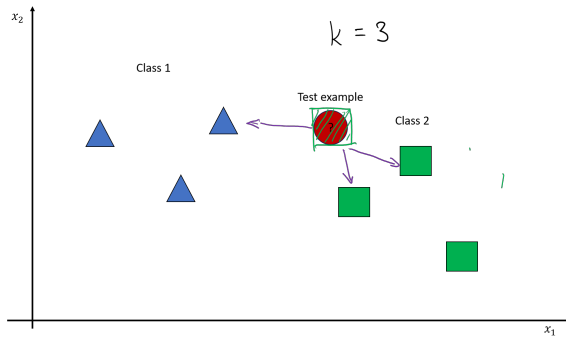


Figure 4: Illustration of the basic idea behind kNNs. In this example the number of nearest neighbours to consider is $k = 3$

Where n is the total number of training examples. Given some new test example, $\mathbf{x}^{[t]}$, and some distance metric, $d(\mathbf{x}^{[a]}, \mathbf{x}^{[b]})$, that measure the distance between two examples¹⁰ We want to sample the k closest training examples (*nearest neighbours*)¹¹ to the test set to produce a new dataset:

$$D_k = \{(\mathbf{x}^{[i]}, \mathbf{y}^{[i]}), \dots, (\mathbf{x}^{[k]}, \mathbf{y}^{[k]})\}$$

Where k is a hyperparameter that is decided upon or tuned¹² to produce optimal results. Once D_k has been obtained one can perform classification or regression.

Classification

The predicted class will be the outcome of a plurality vote¹³ A simple way to describe the hypothesis function is

$$h(\mathbf{x}^{[t]}) = \text{mode}(\{\mathbf{y}^{[i]}, \dots, \mathbf{y}^{[k]}\})$$

Where the mode is the highest occurring label (class) from a given dataset. Or if you want,

$$h(\mathbf{x}^{[t]}) = \underset{j \in \{1, \dots, c\}}{\text{argmax}} \sum_{i=1}^k \delta(j, \mathbf{y}^{[i]})$$

Where c is the number of classes and $\delta(a, b)$ is the Kronecker Delta:

$$\delta(a, b) = \begin{cases} 1, & \text{if } a = b \\ 0, & \text{if } a \neq b \end{cases}$$

The function¹⁴ is used because we want to output the class (label) itself and not the number of target classes that belonged to that class.

Regression

If you wanted to perform regression you could still determine the k nearest neighbours, except instead of calculating the mode of the

¹⁰ remember we can represent examples by a vector of the various features we would like to examine

¹¹ Hence the name k-Nearest Neighbours

¹² Hyperparameter tuning is not the same as learning parameters

¹³ A majority vote traditionally assumes >50% for a single class. This is fine for a two class (binary) scenario, but for multi-class scenarios you want the highest percentage for a given class - not necessarily > 50%

¹⁴ You will come across this function quite frequently when programming. It returns the index of the element with the highest value. The function is similarly used to return the index of the element with the lowest value.

labels you could calculate a real value such as the mean. This would be given by:

$$h(\mathbf{x}^{[t]}) = \frac{1}{k} \sum_{i=1}^k \mathbf{y}^{[i]}$$

Where for regression the labels are real-valued. You could use other functions such as the median etc. Whichever you think would yield the best results.

$n \leftarrow \#$ of training examples $k \leftarrow \#$ of nearest neighbours to consider
 $dist_list \leftarrow []$ $i = 1$ n $d \leftarrow$ distance between $\mathbf{x}^{[t]}$ and $\mathbf{x}^{[i]}$ append d to $dist_list$
 $sorted_indices \leftarrow$ argsort $dist_list$ in ascending order
 choose the k nearest training examples using $sorted_indices$ $D_k \leftarrow D$ sampled using k nearest examples

Manual example of kNN

Let's see this process step by step with a manual example. Suppose we have the classification task illustrated by the points shown in Figure 5.

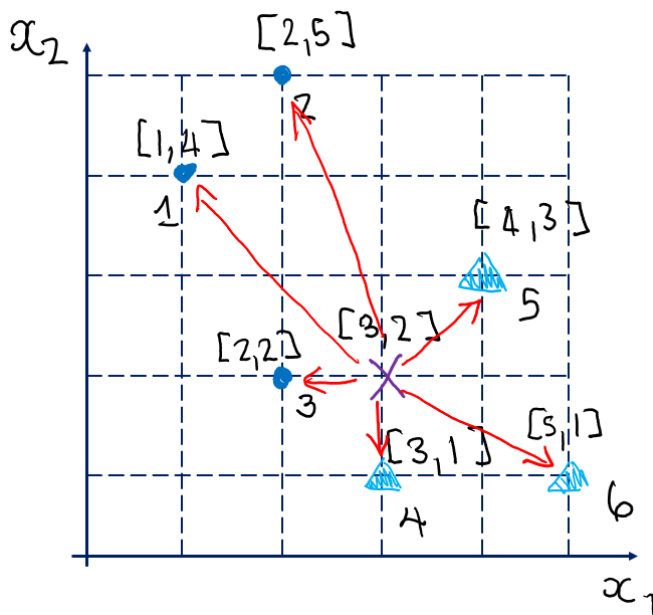


Figure 5: A manual example of how a kNN algorithm works

When classifying a new test example the process is as follows:

1. Calculate the distance from the test example to each training example.
2. Sort in ascending order, and keep track of which example they belong to i.e. indices.

3. Create dataset, D_k by sampling the k nearest examples from, D , with their corresponding classes.
4. Determine the mode of the classes in D_k .
5. The new test example is predicted to belong to this class.

In Figure 5 there are two possible classes. We could say that $y \in \{0, 1\}$. Suppose the blue dots belong to class '1' and the green triangles belong to class '0' ¹⁵ The dataset would be described as:

$$D = \{([1, 4], 1), \dots, ([5, 1], 0)\}$$

Using the numbering conventions shown in the figure to indicate the indices of each training example ¹⁶. The test example is given by $\mathbf{x}^{[t]} = [3, 2]$. Suppose the Euclidean distance is chosen as the distance metric, then the list of all the distances between each training example and the test example would be:

$$dist_list = [2.83, 3.16, 1.0, 1.0, 1.41, 2.23]$$

If this is sorted by distance in ascending order, but the corresponding indices are returned, the result would be:

$$sorted_indices = [3, 4, 5, 6, 1, 2]$$

If $k = 3$ then

$$D_k = \{(\mathbf{x}^{[3]}, \mathbf{y}^{[3]}), (\mathbf{x}^{[4]}, \mathbf{y}^{[4]}), (\mathbf{x}^{[5]}, \mathbf{y}^{[5]})\}$$

The hypothesis function would then yield:

$$\begin{aligned} h(\mathbf{x}^{[t]}) &= mode(\{\mathbf{y}^{[3]}, \mathbf{y}^{[4]}, \mathbf{y}^{[5]}\}) \\ h([3, 2]) &= mode\{1, 0, 0\} \\ &= 0 \end{aligned}$$

This should give you an even clearer idea about how kNNs work.

Decision boundaries

An example of the decision boundaries that could be learned by a kNN algorithm I implemented is shown in Figure 6

The misclassified examples appear reasonable. Some of the misclassified purple test examples appear to be outliers that fall within similar regions of the green class. The lack of separation in the center region intuitively makes classification of those examples difficult.

¹⁵ The classes are arbitrary and in this case indicated by integers. There are other representations for classes that we will come across.

¹⁶ This is completely arbitrary - just make sure the feature vectors and labels correspond

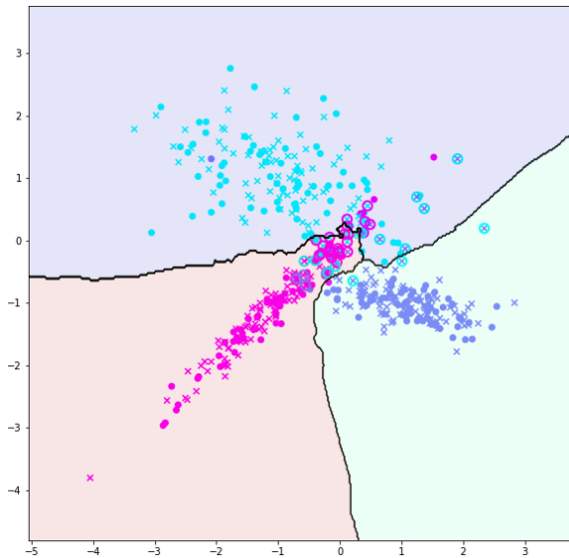


Figure 6: Possible decision boundaries on synthetic data with two features and three possible classes. Solid dots represent training examples. Crosses represent test examples. Circles around crosses indicate misclassified test examples with the colour of the circles indicating the ground truth class

Some problems with kNNs

If you attempted to apply kNNs directly to an image recognition task (see Figure 7) with high-dimensional inputs ¹⁷ the kNN algorithm might struggle. You could potentially reduce the dimensions of the input, for example using the average pixel value across an entire image as one feature. But this would likely cause more issues if there is not enough useful information contained in the reduced number of features ¹⁸.

¹⁷ if each pixel is considered an individual feature the input dimensions are very large.

¹⁸ We will discuss some techniques for feature selection later

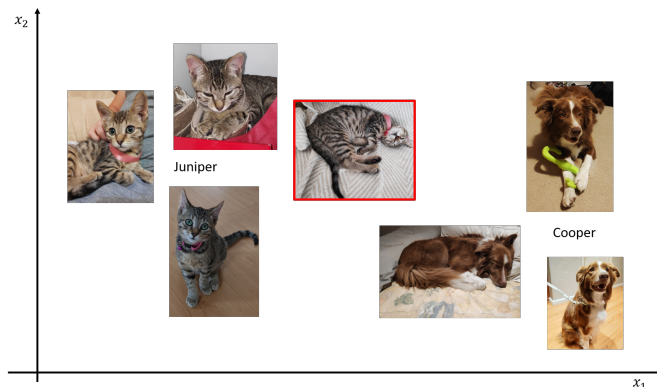


Figure 7: A diagram of an image classification task

kNNs need to store all their training data for use when predicting. This is not always feasible, for example, if you are deploying on an embedded system you might not have access to large amounts of storage or memory.

At this stage I would like to move away from specific algorithms to talk about model evaluation. How do we know if a model will

behave well on unseen data, can we diagnose specific problems and can we solve them.

References

Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An introduction to statistical learning*. Springer, 2017.

Sebastian Raschka. Stat 479: Machine learning lecture notes. <http://pages.stat.wisc.edu/~sraschka/teaching/stat479-fs2019>, 2019.

Sebastian Raschka. What is the difference between a parametric learning algorithm and a nonparametric learning algorithm?, 2020. URL https://sebastianraschka.com/faq/docs/parametric_vs_nonparametric.html.